CHAROTAR UNIVERSITY OF SCIENCE &TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CSPIT

Subject Name: Fundamentals of Database Management Systems **Subject Code**: ITUC202
Semester: IV **Academic Year**: 2025-26

Practical List

Note:

1. Students have to solve the problems in the Oracle platform and Mongo DB platform.
2. Students are instructed to solve challenges given in the Practical List.
3. Internal evaluation of practicals would be based on performance in regular lab sessions and performance in mid-term practical examinations.

Practical-1	Hrs
Aim: A national bank is modernizing its information system to enhance employee management and customer service. The bank requires a well-structured database that can store different job roles (e.g., Clerk, Manager, Loan Officer), along with employees working in each role. At the same time, the bank needs to separate account customers (savings, current, fixed deposit holders) from loan customers (home loan, vehicle loan, personal loan borrowers) to provide better tracking and faster processing. Task is to design the database schema that supports these requirements and implement it using SQL.	4

Constraints –

- Not Null Constraints: Critical fields must not be null to ensure data completeness.
- Unique Constraints: Certain fields must have unique values to avoid duplicates (e.g., Account Number).
- Check Constraints: Enforce domain integrity by limiting the values that can be placed in a column.

Tasks:-

1) Create Table Job (job_id, job_title, min_sal, max_sal)

COLUMN NAME	DATA TYPE
job_id	Varchar2(15)
job_title	Varchar2(30)
min_sal	Number(7,2)
max_sal	Number(7,2)

2) Create table Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no)

COLUMN NAME	DATA TYPE
emp_no	Number(3)
emp_name	Varchar2(30)
emp_sal	Number(8,2)
emp_comm	Number(6,1)

dept_no	Number(3)

3) Create table deposit (a_no, name,bname, amount,a_date).

COLUMN NAME	DATA TYPE
a_no	Varchar2(5)
cname	Varchar2(15)
bname	Varchar2(10)
amount	Number(7,2)
a_date	Date

4) Create table borrow (loan no,cname,bname,amount).

COLUMN NAME	DATA TYPE
loanno	Varchar2(5)
cname	Varchar2(15)
bname	Varchar2(10)
amount	Varchar2(7,2)

5) Insert the following values in the table Employee.

emp_no	emp_name	emp_sal	emp_comm	dept _no
101	Smith	800	455	20
102	Snehal	1600	0	25
103	Adama	1100	425	20
104	Aman	3000		15
105	Anita	5000	50,000	10
106	Anamika	2975		30

6) Insert the following values in the table JOB.

job_id	job_title	min_sal	max_sal
IT_PROG	Programmer	4000	10000
MK_MGR	Marketing manager	9000	15000
FI_MGR	Finance manager	8200	12000
FI_ACC	Account	4200	9000
LEC	Lecturer	6000	17000
COMP_OP	Computer Operator	1500	13000

7) Insert the following values in the table deposit.

a_no	cname	bname	amount	a_date
101	Anil	andheri	7000	01-jan-06
102	sunil	virar	5000	15-jul-06
103	jay	villeparle	6500	12-mar-06
104	vijay	andheri	8000	17-sep-06
105	keyur	dadar	7500	19-nov-06

106	mayur	borivali	5500	21-dec-06

8) Insert the following values in the table borrow.

loanno	cname	bname	amount
201	ANIL	VRCE	1000.00
206	MEHUL	AJNI	5000.00
311	SUNIL	DHARAMPETH	3000.00
321	MADHURI	ANDHERI	2000.00
375	PRMOD	VIRAR	8000.00
481	KRANTI	NEHRU PLACE	3000.00

9) Describe the table Job, employee, deposit, borrow.

Practical-2

2

Aim: To manage and update a company's employee-related database by performing essential Data Definition Language (DDL) operations. In this scenario, the organization is restructuring its data system and needs to create new tables, duplicate existing structures, rename tables, modify column definitions, enforce constraints, and remove unwanted fields. The objective is to ensure that the database schema remains accurate, consistent, and adaptable to organizational changes.

Constraints –

- Not Null Constraints: Ensure critical fields are not null.
- Unique Constraints: Ensure data integrity by limiting column values.
- Check Constraints: Ensure columns like Account Number have unique values.

Tasks:-

1) Create a table supplier from an employee with all the columns and verify.

Test case: Verify suppler table consists of all columns of employee table.

2) Create table sup1 from an employee with the first two columns and verify.

Test case: Verify sup1 consists of first two columns of employee table.

3) Create table sup2 from employee with no data and verify.

Test case: Verify sup2 table contains no data.

4) Insert the data into sup2 from employee whose name is 'Anita' and verify.

Test case: Verify the details of employee whose name is 'Anita' is inserted in sup2 table.

5) Rename the table sup2 and verify.

Test case: Verify the sup2 table name is changed.

6) Destroy table sup1 with all the data and verify.

Test case: Verify table sup1 is destroyed or not.

7) Add one column phone to an employee with size of column is Varchar2(10) and verify.

Test case: Verify phone column is added in employee as per the column size.

8) Modify column phone and change type to char(10) and verify.

Test case: Verify phone column type is changed to char(10).

9) Delete employee_name column from sup2 and verify;

Test case: Verify employee_name column is deleted from sup2.

10) Rename the column salary to new_sal in sup2 and verify.

Test case: Verify the column name of salary is changed to new_sal in sup2 table.

Practical-3

4

Aim: To assist a banking organization in efficiently managing and analyzing their employee, job, and customer-related data by performing essential SQL queries on the existing database schema. The bank requires accurate retrieval of information for decision-making, auditing, and operational activities. In this scenario, the database administrator is tasked with extracting detailed reports such as employee department-wise details, customer deposit information based on date criteria, job salary structures, and identification of specific attributes like non-null commission values

Constraints –

- Not Null Constraints: Ensure critical fields are not null.
- Unique Constraints: Ensure data integrity by limiting column values.
- Check Constraints: Ensure columns have unique values where required.

Test Cases-

1) Retrieve all data from **employee, jobs and deposit.**

Test Case: Verify all rows are retrieved from each table.

2) Display job title and maximum salary of all jobs.

Test Case: Verify the job title and maximum salary for each job.

3) Write a query to find out all the branches.

Test Case: Verify the list of all unique branches.

4) Display all the account no. into which rupees are between dates **01-01-06 and 25-07-06**.

   Test Case: Verify the account numbers with deposits in the specified date range.

5) Display names of all customers whose account is deposited after **09-oct-06**

   Test Case: Verify the names of customers with accounts deposited after the specified date.

6) Display name and salary of employee whose department no is 20. Give alias name to name of employee.

   Test Case: Verify the names and salaries of employees in department 20 with the alias.

7) Display employee no, name and department details of those employee whose department lies **in(10,20).**

   Test Case: Verify the details of employees in departments 10 and 20.

8) Display employee no, name and department details of those employee whose department **not in(15,30) except 25 .**

   Test Case: Verify the details of employees excluding departments 15, 30, and 25.

9) Display employee no, name and department details of those employee whose department no is **between 15 and 25.**

   Test Case: Verify the details of employees in departments between 15 and 25.

10) Display name of all employee whose emp_comm contains the **non-null** values.

    Test Case: Verify the names of employees with non-null emp_comm values.

11) Combine two columns min_sal and max_sal and display it one column using common alias name.
    Test Case: Verify the combined minimum and maximum salary displayed as a single column.

12) Insert the data into sup2 from employee.
    Test Case: Verify the data from Employee is inserted into Supplier2.

Practical-4

4

Aim: To support a banking organization in maintaining accurate, consistent, and meaningful data within their employee and job-related database by performing essential Data Manipulation Language (DML) operations and analytical SQL queries. In this scenario, the bank's database administrator is responsible for updating employee records, deleting outdated or incorrect information, modifying commission values, and generating aggregated insights such as maximum/minimum salary, average salary, and departmental statistics.

The organization also requires well-structured data retrieval, sorted in various orders and filtered by business rules, to enable effective monitoring of employee performance, salary distribution, and department-wise data analysis.

Constraints –

- Not Null Constraints: Ensure critical fields are not null.
- Unique Constraints: Ensure data integrity by limiting column values.
- Check Constraints: Ensure columns have unique values where required.

Test Cases-

1) Delete all the rows from sup1 as sup.

   Test Case: Verify all rows are deleted from Sup1.

2) Delete the detail of supplier whose emp_no is 103.

   Test Case: Verify the row with emp_no 103 is deleted from Supplier.

3) Update the name of employee to **'Aman'** name whose emp_name is 'Anita'.

   Test Case: Verify the name 'Anita' is updated to 'Aman'.

4) Update the value of employee name whose employee number is 103.
   Test Case: Verify the name is updated for the employee with EmployeeID 103.

5) Find out the maximum and minimum salary form job table.
   Test Case: Verify the maximum and minimum salary from the JobProfile table.

6) Find out the average salary of employee.
   Test Case: Verify the average salary of employees.

7) Count the total no as well as distinct rows in dept_no column with a condition of salary greater than 1000 of employee.
   Test Case: Verify the total and distinct count of departments with a salary greater than 1000.

8) Display the detail of all employees in ascending order, descending order of their name and no.
   Test Case: Verify the employee details sorted by name in ascending order and by number in descending order.

9) Display the dept_no in ascending order and accordingly display emp_comm in descending order.
   Test Case: Verify the dept_no in ascending order and emp_comm in descending order.

10) Update the value of emp_comm to 500 where dept_no is 20.
Test Case: Verify the emp_comm is updated to 500 for department 20.

11) Display the emp_comm in ascending order with null value first and accordingly sort employee salary in descending order.
Test Case: Verify emp_comm in ascending order with null values first and salary in descending order.

12) Display the emp_comm in ascending order with null value last and accordingly sort emp_no in descending order.
Test Case: Verify emp_comm in ascending order with null values last and emp_no in descending order.

Practical - 5 | 4

Aim:- To learn and understand the use of mongodb to create and analyze the data.

What is MongoDB

● MongoDB is an open-source document database that provides high performance, high availability, and automatic scaling.

● In simple words, you can say that - Mongo DB is a document-oriented database. It is an open source product, developed and supported by a company named 10gen.

● MongoDB is available under General Public license for free, and it is also available under Commercial license from the manufacturer.

● The manufacturing company 10gen has defined MongoDB as:

● "MongoDB is a scalable, open source, high performance, document-oriented database." - 10gen

● MongoDB was designed to work with commodity servers. Now it is used by the company of all sizes, across all industry.

●

Why Use MongoDB?

● Document Oriented Storage − Data is stored in the form of JSON style documents.

● Index on any attribute

● Replication and high availability

● Auto-Sharding

● Rich queries

● Fast in-place updates

● Professional support by MongoDB

Where to Use MongoDB?

● Big Data

● Content Management and Delivery

● Mobile and Social Infrastructure

● User Data Management

● Data Hub

Mongo Db Official Document

https://www.mongodb.com/docs/mongodb-shell/connect/

Connect to a Local Deployment on the Default Port

To connect to a MongoDB deployment running on localhost with default port 27017, run mongosh without any options:

mongo

OR

mongosh

55

● **To show list of data bases available in the system use following command**

> show dbs

admin 0.000GB

config 0.000GB

local 0.000GB

● **Create a New Database**

use <dbname>

> use product

switched to db product

● **To display the name of current working Database**

> db

product

● **To Create a Collection**

There are two ways to create a collection

1. db.createCollection('books') (db lib)

It will create an empty collection

2. db.users.insert({name:'shital',age:20})(db product)

WriteResult({ "nInserted" : 1 })

● **To Display Collections**

> show collections

product

users

● **To Drop a Collection**

> db.users.drop()

True

● **To Drop a Database**

> db.dropDatabase()

{ "ok" : 1 }

● **To retrieve collection**

1. db.collection.find()

>db.books.find()

2. db.collection.find().pretty()

To pretify the output

>db.books.find().pretty()

● **Insert Documents(insert command)**

The MongoDB shell provides the following methods to insert documents into a collection:

1. **To insert a single document**,

56

use **db.collection.insertOne()** or **db.collection.insert()**.

Insert is deprecated since version 3.0 so instead of that we can use insertOne

> db.books.insert({name:'the compplete reference', page:500, price:1000})

WriteResult({ "nInserted" : 1 })

> db.books.find(){

"_id" : ObjectId("64199568d7fb909707ec850b"), "name" : "the compplete

reference", "page" : 500, "price" : 1000 }

> db.books.insertOne({name:'the python', page:600, price:800})

{

"acknowledged" : true,

"insertedId" : ObjectId("641995c2d7fb909707ec850c")

}

> db.books.find()

{ "_id" : ObjectId("64199568d7fb909707ec850b"), "name" : "the compplete reference", "page" : 500, "price" : 1000 } { "_id" : ObjectId("641995c2d7fb909707ec850c"), "name" : "the python", "page" : 600, "price" : 800 } 2. **To insert multiple documents** use db.collection.insertMany(). db.books.insertMany([{name:'the compplete reference', page:500, price:1000},{name:'the python programming', page:800, price:1500},{name:'software engineering', page:500, price:2000}) ● **Update Documents(Update command)** **The MongoDB shell provides the following methods to update documents in a collection:** ● **To update a single document, use db.collection.updateOne(). Or update()** db.books.update({"_id" : ObjectId("64199568d7fb909707ec850b")}, ... {name:'new one'}) WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 }) db.books.updateOne({_id:ObjectId("64199568d7fb909707ec850b")}, {name:"the complete reference",pages:8000, price:5000}) ● **To update multiple documents, use db.collection.updateMany().** db.books.updateMany({_id:ObjectId("64199568d7fb909707ec850b")}, {name:"the complete reference",pages:8000, price:5000}) ● To replace a document, use db.collection.replaceOne(). ● **Update Operators** The following modifiers are available for use in update operations https://www.mongodb.com/docs/manual/reference/operator/update/ specify the operator expression in a document of the form: 57 { <operator1>: { <field1>: <value1>, ... }, <operator2>: { <field2>: <value2>, ... }, ... ● **Update the document with specific Id** > db.books.updateOne({_id:ObjectId("64199568d7fb909707ec850b")}, {$set:{name:"the complete reference 2",pages:8000, price:5000}}) { "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }	

● **Update the name to bookname for specific Id** db.books.updateOne({_id:ObjectId("64199568d7fb909707ec850b")}, {$rename:{name:"bookname"}}) ● **Update the document name to book name where pages=8000** db.books.update({pages:8000}, {$rename:{name:"book_name"}}) ● **Update all the documents name to book name where price=800** db.books.updateMany({price:800}, {$rename:{name:"book_name"}}) { "acknowledged" : true, "matchedCount" : 3, "modifiedCount" : 2 } ● **Add 5 in price where price=800** db.books.update({price:800}, {$inc:{price:5}}) **Delete Document(delete command)** The MongoDB shell provides the following methods to delete documents from a collection: ● **To delete multiple documents, use db.collection.deleteMany().** Delete all those documents which is having title= Titanic use sample_mflix db.movies.deleteMany({ title: "Titanic" }) ● **To delete a single document, use db.collection.deleteOne(). Or remove()** ● **remove()** **Remove the document with specific Id** db.books.remove({_id:ObjectId("64199568d7fb909707ec850b")}) ● **deleteOne()** use sample_mflix 58 Remove the document where cast= Brad Pitt db.movies.deleteOne({ cast: "Brad Pitt" })	
Practical - 6	**4**
Query Document (Query and projection Operator) **1. db.collection.find()** **To get all the documents from a collection** db.books.find() **2. db.collection.find().pretty()** **To pretify the output** db.books.find().pretty()	

3. Get 2 record from a collection

db.books.find().limit(2)

1. Sort data based on values

db.books.find().sort({price:1}).pretty()

Note :- the above query will sort the data based on price

1= asc

-1= desc

1. Pipeline Functions

display the details of 3 books sorted by desc order

db.books.find().limit(3).sort({name:-1}).pretty()

skip()- skip the number of documents

2. Using Query Operator in querying the document

● **Get all those books whose price =800**

db.books.find({price:{$eq:800}})

● **Get all books where price>800**

db.books.find({price:{$gt:800}})

● **Get all the books where price is in 800, 900(array)**

db.books.find({price:{$in:[800,900]}})

● **Get all the books whose price=800 and pages=600**

db.books.find({$and:[{page:600},{price:800}]})

● **Do not retrieve those books whose price is not greater then 800**

db.books.find({price:{$not:{$gt:800}}})

4. Projection:

Projection is used to project the number of columns or attributes from a collection

- Use following syntax

{attname:1/0}

1= True

0= False

● **Get all booknames**

59

> db.books.find({},{book_name:1})

● **Get all booknames with Id whose price=805**

> db.books.find({price:805},{book_name:1})

{ "_id" : ObjectId("641995c2d7fb909707ec850c"), "book_name" : "the python" }

● **Get all booknames whose price=805**

> db.books.find({price:805},{book_name:1,_id:0})

{ "book_name" : "the python" }

5. Get The total number of collections

db.books.find().count()

6. MongoDb with mapreduce

Consider a stud collection with following data

db.stud.find({})

{ "_id" : ObjectId("64e05c5f3a2012e8186751eb"), "id" : 1, "sec" : "A", "marks" : 80 }

{ "_id" : ObjectId("64e05c9c3a2012e8186751ec"), "id" : 2, "sec" : "A", "marks" : 90 }

{ "_id" : ObjectId("64e05cb23a2012e8186751ed"), "id" : 1, "sec" : "B", "marks" : 99 }

{ "_id" : ObjectId("64e05ce83a2012e8186751ee"), "id" : 1, "sec" : "B", "marks" : 95 }

{ "_id" : ObjectId("64e05cfe3a2012e8186751ef"), "id" : 1, "sec" : "C", "marks" : 90 }

1. Define the map function

2. Define the reduce function

3. Use the mapReduce function by passing mapper function, reducer function and the output collection name

60

Example :-

Exercise:

1. Write the insert method to store the following document in MongoDB and verify. May practice

crud operations.

Name : "Stephen More"

Address : {

"City" : "Banglore",

"Street" : "Electronics City",

"Affiliation" : "XYZ Ltd"

}

Hobbies : Chess, Lawn Tennis, Base Ball

2. Create a collection 'cities' as follows

```
db.cities.insertMany([ {"name": "Seoul", "country": "South Korea", "continent": "Asia",
"population":
25.674 },
{"name": "Mumbai", "country": "India", "continent": "Asia", "population": 19.980 },
{"name": "Lagos", "country": "Nigeria", "continent": "Africa", "population": 13.463 },
{"name": "Beijing", "country": "China", "continent": "Asia", "population": 19.618 },
{"name": "Shanghai", "country": "China", "continent": "Asia", "population": 25.582 },
{"name": "Osaka", "country": "Japan", "continent": "Asia", "population": 19.281 },
{"name": "Cairo", "country": "Egypt", "continent": "Africa", "population": 20.076 },
{"name": "Tokyo", "country": "Japan", "continent": "Asia", "population": 37.400 },
{"name": "Karachi", "country": "Pakistan", "continent": "Asia", "population": 15.400 },
{"name": "Dhaka", "country": "Bangladesh", "continent": "Asia", "population": 19.578 },
{"name": "Rio de Janeiro", "country": "Brazil", "continent": "South America", "population": 13.293
},
61
{"name": "São Paulo", "country": "Brazil", "continent": "South America", "population": 21.650 },
{"name": "Mexico City", "country": "Mexico", "continent": "North America", "population": 21.581
},
{"name": "Delhi", "country": "India", "continent": "Asia", "population": 28.514 },
{"name": "Buenos Aires", "country": "Argentina", "continent": "South America", "population":
14.967 },
{"name": "Kolkata", "country": "India", "continent": "Asia", "population": 14.681 },
{"name": "New York", "country": "United States", "continent": "North America", "population":
18.819 },
{"name": "Manila", "country": "Philippines", "continent": "Asia", "population": 13.482 },
{"name": "Chongqing", "country": "China", "continent": "Asia", "population": 14.838 },
{"name": "Istanbul", "country": "Turkey", "continent": "Europe", "population": 14.751 }
])
```

- Write a Mongodb query to find all the cities of North America.
- Write a mongodb query to retrieve the cities names in descending order of their population.
- Write a mongodb query to display all the cities grouped by their continent.
- Write a mongodb query to add the field “highest population”.

3. To practice MapReduce programming in MongoDB.

Step. 3.1: Insert 5 documents as shown below in the collection named 'books'.

```
> db.books.find({}).pretty();
{
"_id" : 1,
"Category" : "Machine Learning",
"BookName" : "Machine Learning for Hackers",
"Author" : "Drew Conway",
"qty" : 25,
"price" : 400,
"rol" : 30,
"pages" : 350
}{
"_id" : 2,
"Category" : "Business Intelligence",
"BookName" : "Fundamentals of Business Analytics",
"Author" : "Seema Acharya",
"qty" : 55,
Page 6 of 23"price" : 500,
"rol" : 30,
"pages" : 250
}{
"_id" : 3,
"Category" : "Analytics",
"BookName" : "Competing on Analytics",
"Author" : "Thomas Davenport",
"qty" : 8,
"price" : 150,
62
"rol" : 20,
"pages" : 150
}{
"_id" : 4,
```

"Category" : "Visualization",
"BookName" : "Visualizing Data",
"Author" : "Ben Fry",
"qty" : 12,
Page 7 of 23"price" : 325,
"rol" : 6,
"pages" : 450
}{
"_id" : 5,
"Category" : "Web Mining",
"BookName" : "Learning R",
"Author" : "Richard Cotton",
"qty" : 5,
"price" : 850,
"rol" : 10,
"pages" : 120
}
Step. 3.2: Confirm the presence of the above documents in the “books” collection.
Step. 3.3: Write map and reduce functions to split the books into the following two categories:
(a) Big Books
(b) Small Books
Books which have more than 300 pages should be in the big book category. Books which have less than 300 pages should be in the small book category.
Step. 3.4: Count the number of books in each category.
Step 3.5: Store the output as follow as documents in a new collection, called “Book_Result”.
Book Category Count of the books
“Big Books" 2
"Small Books" 3
3.Write a mongodb query using mapreduce to find mutual friends.
-you may consider the required collections.	

Practical - 7	**4**
Aim: To solve various queries related to grouping and aggregate functions by manipulating data in the product and emp_company tables.	

Constraints –

- Not Null Constraints: Ensure critical fields are not null.
- Unique Constraints: Ensure data integrity by limiting column values.
- Check Constraints: Ensure columns have unique values where required.

Test Cases –

1) Insert the following values into the product table.

Detorder _no	Product_no	Qty_order
O19001	P00001	10
O19001	P00002	3
O19002	P00001	4
O19003	P00004	2
O19004	P00003	6
O19005	P00005	2
O19006	P00004	7

Test Case: Verify that the values are inserted correctly into the product table.

2) Retrieve the product numbers and total quantity ordered for each product from the product table.

   Test Case: Verify that the sum of quantities ordered for each product number is calculated correctly.

3) Retrieve the product no and the total quantity ordered for product's 'P00001' and 'P00004' from product table.

   Test Case: Verify that the sum of quantities ordered for product numbers 'P00001' and 'P00004' is calculated correctly.

4) Insert the following values into emp_company

ENAME	CNAME	SALARY
Anil	ACC	1500
Shankar	TATA	2000
Jay	WIPRO	1800
Sunil	WIPRO	1700
Vijay	TATA	5000
Prakash	TATA	3000

Ajay	ACC	8000
Abhay	ACC	1800

Test Case: Verify that the values are inserted correctly into the emp_company table.

5) List the name of the company and the maximum salary in that company.

   Test Case: Verify that the maximum salary for each company is calculated correctly.

6) Find out the average salary of each company.

   Test Case: Verify that the average salary for each company is calculated correctly.

7) Find out the names of companies having an average salary of more than 1500.

   Test Case: Verify that the companies with an average salary greater than 1500 are listed correctly.

8) Find out the average salary of each company except 'ACC'.

Test Case: Verify that the average salary for each company, excluding 'ACC', is calculated correctly.

Practical-8 | 2

AIM : For a given University schema, create tables and generate a Master-Slave relationship along with all the necessary integrity constraints in MS Access Tool.

Practical-9 | 4

AIM - To apply the concept of integrity/data constraints while creating or altering tables in a database for managing sales-related data.

1) Create Table Salespeople where Snum number(4) P.K, Sname varchar2(20) NOT NULL, City varchar2(15), Comm number(5,2).

Constraints

1. **Primary Key Constraint**: Ensure Snum uniquely identifies each record.
2. **Not Null Constraint**: Ensure Sname is not null.

Tasks:

Test Case 1: Insert a valid record.

Objective: Verify that a valid record can be inserted into the Salespeople table.

Expected Result: The record should be inserted successfully.

Test Case 2: Insert a record with a duplicate Snum

Objective: Verify that inserting a record with a duplicate Snum results in an error.

Expected Result: The insertion should fail with a primary key constraint violation error.

Test Case 3: Insert a record with a null Sname

Objective: Verify that inserting a record with a null Sname results in an error.

Expected Result: The insertion should fail with a not-null constraint violation error.

Test Case 4: Insert a record with missing City and Comm

Objective: Verify that a record can be inserted with City and Comm as null.

Expected Result: The record should be inserted successfully with City and Comm as null.

Test Case 5: Retrieve all records from the Salespeople table.

Objective: Verify that all records can be retrieved from the table.

2) Create Table customer where Cnum number (4) P.K, Cname varchar2(20)NOT NULL, City varchar2(20), Rating number(3) DEFAULT 10, Snum number(4) F.K.(where snum refer salespeople table).

Constraints

1. **Primary Key Constraint**: Ensure Cnum uniquely identifies each record.
2. **Not Null Constraint**: Ensure Cname is not null.
3. **Foreign Key Constraint**: Ensure Snum references the Snum field in the Salespeople table.
4. **Default Constraint**: Ensure Rating defaults to 10 if no value is provided.

Tasks:

Test Case 1: Insert a valid record

Objective: Verify that a valid record can be inserted into the Customer table.

Expected Result: The record should be inserted successfully.

Test Case 2: Insert a record with a missing Rating

Objective: Verify that a record can be inserted with a Rating defaulting to 10.

Expected Result: The record should be inserted successfully with a Rating of 10.

Test Case 3: Insert a record with a null Cname

Objective: Verify that inserting a record with a null Cname results in an error.

Expected Result: The insertion should fail with a not null constraint violation error.

Test Case 4: Insert a record with a non-existent Snum

Objective: Verify that inserting a record with a non-existent Snum results in an error.

Expected Result: The insertion should fail with a foreign key constraint violation error.

Test Case 5: Insert a record with missing City and Rating

Objective: Verify that a record can be inserted with City and Rating as null/default.

Expected Result: The record should be inserted successfully with City as null and Rating as 10.

Test Case 6: Retrieve all records from the Customer table

Objective: Verify that all records can be retrieved from the table.

Expected Result: The query should return all the inserted records.

3) Create table Order where Order_no number(4) P.K, Amount number(5), Odate varchar2(10), Cnum number(4) F.K, (where cnum refer customer table). Snum number(4) F.K (where snum refers salespeople table).

Constraints

1. **Primary Key Constraint**: Ensure Order_no uniquely identifies each record.
2. **Foreign Key Constraint**: Ensure Cnum references the Cnum field in the Customer table.
3. **Foreign Key Constraint**: Ensure Snum references the Snum field in the Salespeople table.

Tasks:

Test Case 1: Insert a valid record

Objective: Verify that a valid record can be inserted into the Orders table.

Expected Result: The record should be inserted successfully.

Test Case 2: Insert a record with a non-existent Cnum

Objective: Verify that inserting a record with a non-existent Cnum results in an error.

Expected Result: The insertion should fail with a foreign key constraint violation error for Cnum.

Test Case 3: Insert a record with a non-existent Snum

Objective: Verify that inserting a record with a non-existent Snum results in an error.

Expected Result: The insertion should fail with a foreign key constraint violation error for Snum.

Test Case 4: Insert a record with a null Amount

Objective: Verify that a record can be inserted with Amount as null.

Expected Result: The record should be inserted successfully with Amount as null.

Test Case 5: Insert a record with null Odate

Objective: Verify that a record can be inserted with Odate as null.

Expected Result: The record should be inserted successfully with Odate as null.

Test Case 6: Retrieve all records from the Orders table

Objective: Verify that all records can be retrieved from the table.

Expected Result: The query should return all the inserted records.

4. Table: Sales_order

Constraints

- **Primary Key Constraints**: Ensure unique identification of records.
- **Foreign Key Constraints**: Maintain relationships between tables.
- **Not Null Constraints**: Ensure critical fields are not null.
- **Check Constraints**: Ensure data integrity by limiting column values.
- **Unique Constraints**: Ensure columns have unique values where required.
- **Default Constraints**: Assign default values to columns when no value is provided.

Column Name	Data Type	Constraints
Order_no	Varchar2(6)	Primary key/First letter starting with 'O'
Order_date	Date	
Client_no	Varchar2(6)	Foreign Key references client_no of client_master table
Dely_addr	Varchar2(25)	
Salesman_no	Varchar2(6)	Foreign Key references sales_no of Salesman_master table
Dely_type	char	Delivery: part(P)/ full (f) Default 'F'

Order_status	Varchar2	In Process, Fulfilled, Backorder, Cancelled

Test Case: Verify that all constraints, including the primary key, foreign key, check constraints, and default values, are correctly applied.

5. Table: salesman_master

Constraints

- **Primary Key Constraints**: Ensure unique identification of records.
- **Foreign Key Constraints**: Maintain relationships between tables.
- **Not Null Constraints**: Ensure critical fields are not null.
- **Check Constraints**: Ensure data integrity by limiting column values.
- **Unique Constraints**: Ensure columns have unique values where required.
- **Default Constraints**: Assign default values to columns when no value is provided.

Column name	**Data type**	**Constraints**
Salesman_no	Varchar2(6)	The primary key/first letter must start with 'S'
Salesman _name	Varchar2(20)	Not null
Address	Varchar2(30)	Not null
City	Varchar2(20)	
Pincode	Varchar2(8)	
State	Varchar2(20)	
Sal_amt	Number(8,2)	Not null, cannot be 0
Ytd_sales	Number(6,2)	Not null, cannot be 0
Tgt_sales	Number(6,2)	Not null

Test Case: Verify that all constraints, including the primary key, not null, and check constraints, are correctly applied.

6. Table: Client_master

Constraints

- **Primary Key Constraints**: Ensure unique identification of records.
- **Foreign Key Constraints**: Maintain relationships between tables.
- **Not Null Constraints**: Ensure critical fields are not null.
- **Check Constraints**: Ensure data integrity by limiting column values.
- **Unique Constraints**: Ensure columns have unique values where required.
- **Default Constraints**: Assign default values to columns when no value is provided.

Column name	Data type	Constraints
Client_no	Varchar2(6)	The primary key/first letter must start with 'C'
Name	Varchar2(20)	Not null
Address	Varchar2(30)	
City	Varchar2(15)	
Pincode	Number(8)	
State	Varchar2(15)	
Bal_due	Number(10,2)	

Test Case: Verify that all constraints, including the primary key, not null, and check constraints, are correctly applied.

Practical-10

4

Aim: To study and execute various JOIN commands to perform data retrieval and manipulation from Salespeople, Customer, and Order tables based on specific requirements.

Constraints

1. **Primary Key Constraints**: Ensure unique identification of records in each table.
2. **Foreign Key Constraints**: Maintain referential integrity between related tables.
3. **Not Null Constraints**: Ensure essential fields are not left empty.

Salespeople:

snum sname city comm.

1001 Peel London 0.12

1002 Serres San Jose 0.13

1004 Motika London 0.11

1007 Rifkin Barcelona 0.15

1003 Axelord New York 0.10

Customer:

cnum cname city rating snum

2001	Hoffman	London	100	1001
2002	Giovanne	Rome	200	1003
2003	Liu	San Jose	300	1002
2004	Grass	Berlin	100	1002
2006	Clemens	London	300	1007
2007	Pereira	Rome	100	1004

Order:

onum	amt	odate	cnum	snum
3001	18.96	10.3.94	2002	1002
3003	767.19	10.3.94	2001	1001
3002	1900.10	10.3.94	2007	1003
3005	5160.45	10.3.94	2003	1002
3006	1098.16	10.3.94	**2008**	1002
3009	1713.23	10.4.94	2002	1003
3007	75.75	10.4.94	2004	1002
3008	4723.95	10.5.94	2006	1001
3010	1309.95	10.6.94	2004	1002
3011	9891.00	10.6.94	2006	1001

Test Cases -

1. All customers serviced by Peel or Motika

Expected Result: List of customers serviced by salespeople named Peel or Motika.

2. All orders except those with 0 or null value in the amt field

Expected Result: List of all orders where amt is not 0 or null.

3. Largest order taken by each sales order value of more than 3000

Expected Result: Largest order amount for each salesperson where the order value is more than 3000.

4. All combinations of salespeople and customers who belong to the same city

Expected Result: List of salespeople and customers who are from the same city.

5. List each order with the name of the customer who placed the order

Expected Result: List of orders with the corresponding customer names who placed the orders.

6. Produce a listing of all the customers serviced by salespeople having a commission of more than 12%

Expected Result: List of customers serviced by salespeople with a commission greater than 12%.

7. Produce names and cities of all customers with the same rating as Hoffman

Expected Result: List of customers who have the same rating as Hoffman.

8. Count the customers with ratings above San Jose's average

Expected Result: Number of customers with a rating above the average rating of customers from San Jose.

9. Find the total amount in orders for each salesperson for whom this total is greater than the amount of the largest order in the order table

Expected Result: Total amount of orders for each salesperson where the total is greater than the largest single order amount in the orders table.

10. Create a union of two queries that shows all customers' names, cities, and ratings. Those with a rating of 200 or greater will have the words 'High Rating' while others will have 'Low Rating'.

Expected Result: List of customers with their names, cities, ratings, and a rating category indicating 'High Rating' or 'Low Rating'.

Practical-11 | 4

Aim: To create a PL/SQL procedure that performs the multiplication of two numbers. The procedure will take two input parameters and return the product.

Constraints

1. **Input Parameters**: The procedure will accept two input parameters of type NUMBER.
2. **Output Parameter**: The procedure will have an output parameter to return the result of the multiplication.

3. **Error Handling**: The procedure will handle potential errors such as invalid input types.

Tasks -

Test Case 1: Multiplying two positive numbers

Input: num1 = 10, num2 = 5

Expected Output: The result should be 50.

Test Case 2: Multiplying a positive number and zero

Input: num1 = 15, num2 = 0

Expected Output: The result should be 0.

Test Case 3: Multiplying two negative numbers

Input: num1 = -4, num2 = -6

Expected Output: The result should be 24.

Test Case 4: Multiplying a positive number and a negative number

Input: num1 = 7, num2 = -3

Expected Output: The result should be -21.

Test Case 5: Multiplying two floating-point numbers

Input: num1 = 2.5, num2 = 4.2

Expected Output: The result should be 10.5.

Practical-12 | 4

Aim: To create a PL/SQL block that deletes records from a table with age 21. Additionally, it generates a trigger that stores the original record in another table before deletion.

Constraints

1. **Deletion Condition**: Delete records from the main table where the age is 21.
2. **Trigger**: Before deleting a record, store the original record in another table.

Tasks -

Test Case 1: Delete records where the age is 21

Objective: Verify that records with an age of 21 are successfully deleted. **Expected Result**: Records with an age of 21 ('Alice' and 'Charlie') should be deleted. **Test Case 2**: Verify Trigger Functionality **Objective**: Verify that the trigger successfully stores original records in the DeletedRecords table before deletion. **Expected Result:** The DeletedRecords table should contain the original records of 'Alice' and 'Charlie' before they were deleted.	